# Advanced Patch Notes

# Satellite Tracker

# Version 3.0

Jun 2026

**Author:**
Felix Meyer
felix.meyer@uzh.ch

# Contents

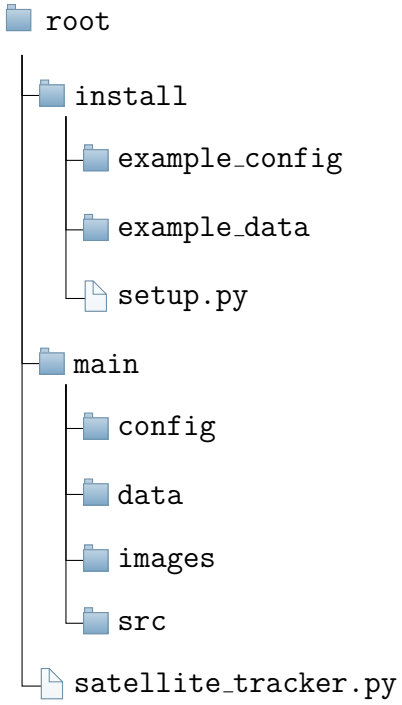
# 1 About this Document

I wrote version 1 of this app for my bachelor's thesis at the University of Zurich. This document, however, should not be understood as a scientific publication but rather as a hands-on manual/patch note for version 3 of the Satellite Tracker App. Version 3 is a complete rewrite of version 1, from the ground up.[1] However, the changes mostly focus on making the code more readable and easier to work with, as well as general improvements in reliability and efficiency. Therefore, I will not explain the core concepts again. I will assume that you are familiar with the thesis and will only explain what has changed. In section 2, I will focus on the interface and major changes, while in section 3, I will give a more general overview on how to navigate through the code, which is now distributed over several files.

# 2 Patch Notes

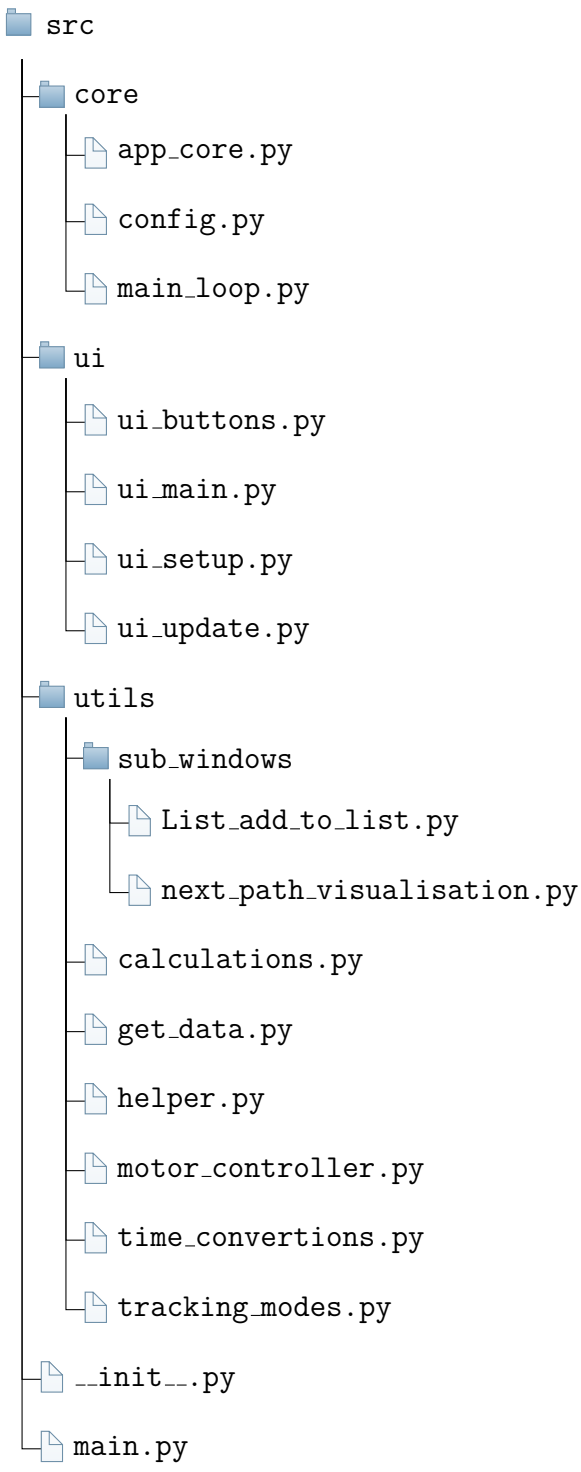
## 2.1 Project File Structure

We now have an install folder containing empty/example versions of files, which we are not tracking on GitHub. By running `setup.py`, they get copied and renamed to where they belong.

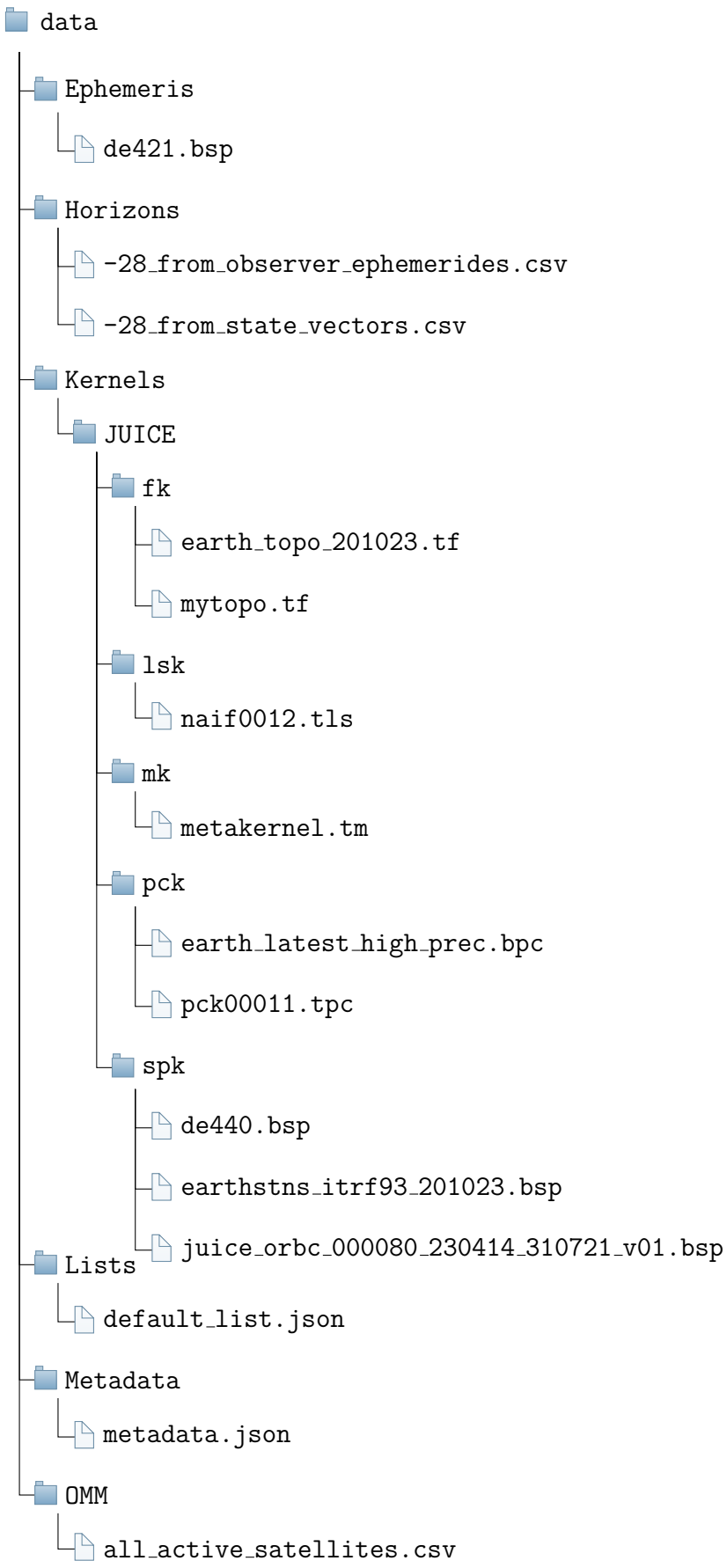



Since the code got split up and distributed over several files, we collect them in `main/src/`.

[1] We don't talk about what happened to version 2.




In `data/`, we have some minimal changes, mostly renaming and reorganizing. I will discuss them in more detail in the next sections.


data
Ephemeris
de421.bsp
Horizons
-28_from_observer_ephemerides.csv
-28_from_state_vectors.csv
Kernels
JUICE
fk
earth_topo_201023.tf
mytopo.tf
lsk
naif0012.tls
mk
metakernel.tm
pck
earth_latest_high_prec.bpc
pck00011.tpc
spk
de440.bsp
earthstns_itrf93_201023.bsp
juice_orbc_000080_230414_310721_v01.bsp
Lists
default_list.json
Metadata
metadata.json
OMM
all_active_satellites.csv

### 2.1.1 Data

**Horizons**

Now contains two files per spacecraft. `ID_from_observer_ephemerides.csv` contains data that we get from Horizons directly, while `ID_from_state_vectors.csv` contains data that we calculate from state vectors.

**OMM**

What was CelesTrack in version 1 is now called OMM. The reason is that the old Two Line Elements (TLE) format will be deprecated when it runs out of 5-digit catalog numbers at 69999 (not 99999), which is estimated to occur around 12.07.2026. Therefore, we no longer support TLEs and only use the modern format OMM.

**Kernels**

The data needed to use SPICE is now in a subfolder for each spacecraft. This needs to be reflected in the metakernel accordingly.

```
\begindata

PATH_VALUES  = ( 'Main/data/Kernels' )

PATH_SYMBOLS = ( 'KERNELS' )

KERNELS_TO_LOAD = (
    '$KERNELS/JUICE/fk/earth_topo_201023.tf'
    '$KERNELS/JUICE/fk/mytopo.tf'
    '$KERNELS/JUICE/lsk/naif0012.tls'
    '$KERNELS/JUICE/pck/earth_latest_high_prec.bpc'
    '$KERNELS/JUICE/pck/pck00011.tpc'
    '$KERNELS/JUICE/spk/de440.bsp'
    '$KERNELS/JUICE/spk/earthstns_itrf93_201023.bsp'
    '$KERNELS/JUICE/spk/juice_orbc_000080_230414_310721_v01.bsp'
)

\begintext
```

**List**

The list moved from the config folder into its own folder within `data/`. The reason is that we can now have multiple lists. However, `default_list.json` must always exist. The format for the list has changed as well. Now we have three target types: Low

Earth Orbit (LEO). These are targets with OMM/CelesTrack data; Deep Space (DS). These are targets with Horizons data; and Astronomical (ASTRO). These are targets with RA/DEC coordinates. In version 1 ASTRO targets could only be targeted in tracking mode RA/DEC. Now they are part of the list. Also, we now no longer have the international designator (Int'l) for LEO targets.


```
[
    {
        "type": "LEO",
        "name": "ISS (ZARYA)",
        "frequency": 145.825,
        "NORAD": 25544
    },
    {
        "type": "DS",
        "name": "Lunar Reconnaissance Orbiter",
        "frequency": 0.0,
        "Horizons": -85
    },
    {
        "type": "ASTRO",
        "name": "Messier 51 (Whirlpool-Galaxy)",
        "frequency": 0.4,
        "RA": 13.4979722,
        "DEC": 47.1952778
    }
]
```


**Metadata**

The metadata file is now in its own folder and has this format:


```
{
    "OMM": {
        "last download": "2026-06-28T09:23:30.017906+00:00"
    },
    "DS": {
        "-28": {
            "last download": "2026-06-26T09:18:25.955004+00:00",
            "valid until": "2026-06-27T09:17:11.816000"
        }
```

```
    }
}
```

### 2.1.2 Config

The config file is now split up into two files:

```
------------------------- config_antenna.env -------------------------
LATITUDE = 0.0
LONGITUDE = 0.0
ALTITUDE = 0.0
LOCAL_TZ = Your_Time_Zone
IP_ADRESS = 0.0.0.0
PORT = 0
CORRECTION_ROLL  = 0.0
CORRECTION_PITCH = 0.0
CORRECTION_YAW   = 0.0

----------------------------- config_app.env -----------------------------
MIN_ANGLE_CHANGE_BEFORE_UPDATE = 0.5
MIN_BEFORE_RECALCULATING_GROUND_TRACK = 0
GROUND_TRACK_STEPS = 90
AUTO_UNCHECK_START_TRACKING_AT_AOS_BTN = True
TIME_RESOLUTION_HORIZONS_STATE_VECTOR = 1
TIME_RESOLUTION_HORIZONS_DIRECTLY = 1
```

## 2.2 Getting Data and Loading Data

During the Artemis II mission, we noticed a major difference between the data we calculated from state vectors and the numbers that the Horizons website was showing. Therefore, I rewrote the whole calculation from the ground up. This time we use `astropy` instead of `Skyfield`. `astropy` is much more precise than `Skyfield` and unlike `Skyfield` it is actually made for Deep Space.

The difference is now less than 0.8°. This is still significant, and the difference is periodic with a period of one day (cf. Section 4.6). That makes me think that there is still some systematic error. Therefore, in version 3, we download 2 datasets.

The first is the same set of state vectors that we have downloaded in version 1. This time, however, we use `astroquery` to send the request instead of writing our own HTTP request.

In version 1, we calculated azimuth, elevation, etc... in the main loop at run time. In version 3, we make use of vectorization in order to do all calculations at once after downloading them. This way, we don't save the raw state vector data. Instead, we can save the computed values that we are actually interested in. We save this as `ID_from_state_vectros.csv`.

In version 1, we added the whole dataset as a `DataFrame` to the list in memory. In version 3, we use `scipy.interpolate.interp1d` and simply add an interpolator to the list in memory. This way, we get our values almost instantaneously even though `astropy` is more computationally expensive by a factor of 10 than `Skyfield`.

For the second dataset, we send Horizons the position of our antenna and tell it to do the calculations for us. We then save that data as `ID_from_observer_ephemerides.csv` and add an interpolator to the list in memory.

One might ask, why then do we even need the first dataset? For two reasons. Firstly, we want to retain the idea of independence from Horizons, just in case it is no longer available in the future.

Secondly, we can't get the data for the subpoint directly from Horizons. We can only calculate it from the state vector data. So, when tracking a Deep Space target, we will use the data that we get from Horizons directly over the data calculated from state vectors for everything except the subpoint.

## 2.3 Interface and Tracking Modes

### 2.3.1 Status

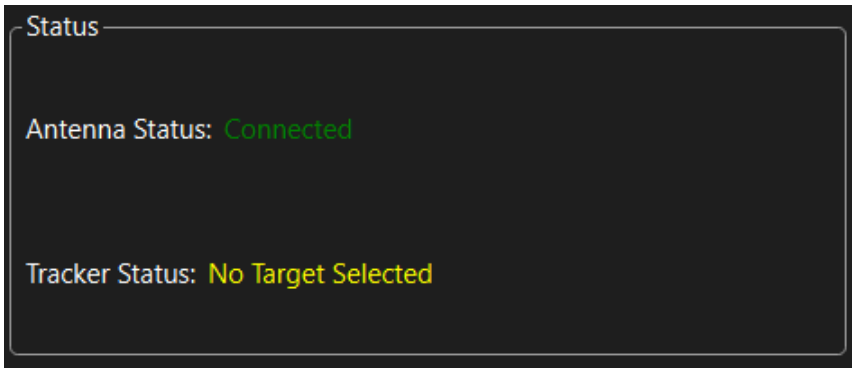



Figure 1: Antenna Status: Connected (green), Not Connected (red); Tracker Status: Tracking (green), Waiting for AOS (green), No Target Selected (yellow), ERROR (red)

A new feature in version 3 is the status indicator. The idea behind this widget is that it should show the user at a glance if the antenna and tracking software are configured correctly for the next observation. The antenna has two states:

- Connected (green)

- Not Connected (red)

The tracking software has four states:

- Tracking (green)
- Waiting for AOS (green), `Start Tracking at AOS` selected but not tracking
- No Target Selected (yellow), neither `Start Tracking at AOS` selected, nor tracking
- ERROR (red)

### 2.3.2 N/A

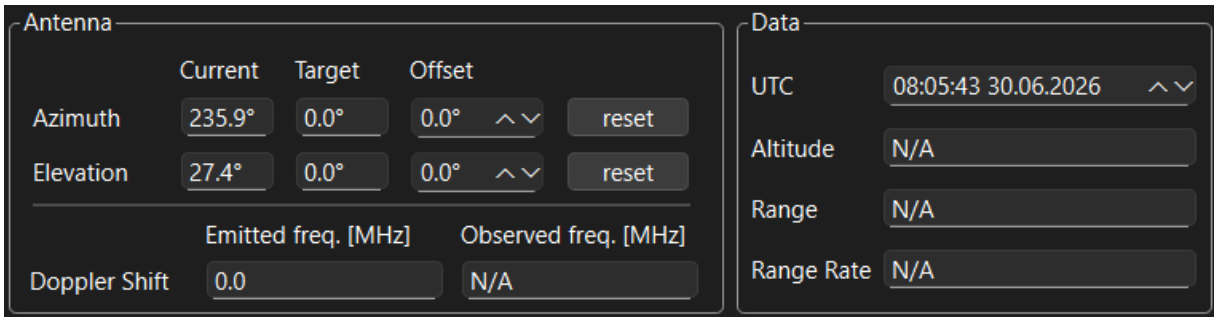



Figure 2: Showing N/A instead of 0 for values that don't get calculated.

We now show N/A instead of 0 when a value isn't calculated. Otherwise, the user might be under the impression that the value gets calculated but is 0.

### 2.3.3 Tracking Mode List

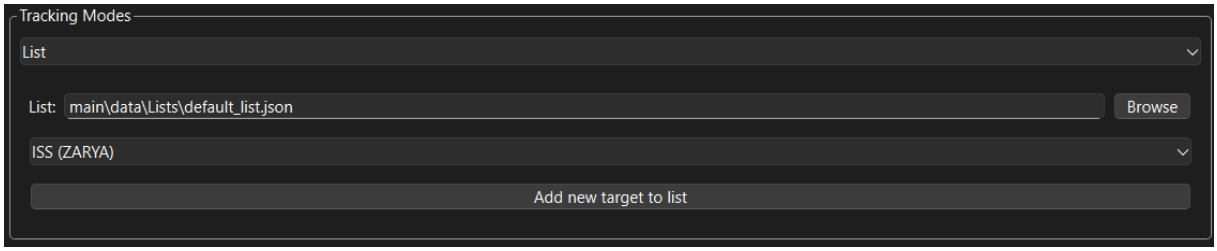



Figure 3: Reworked interface for tracking mode List

The tracking mode List got reworked. As mentioned before, we can now have astronomical targets on the list as well, and it is now possible to have multiple lists. However, `main/data/Lists/default_list.json` must always exist and contain a valid list.[2]

We can now also add new targets to the list via an interface instead of editing the JSON file (cf. Figure 4).

[2]The list can be empty.

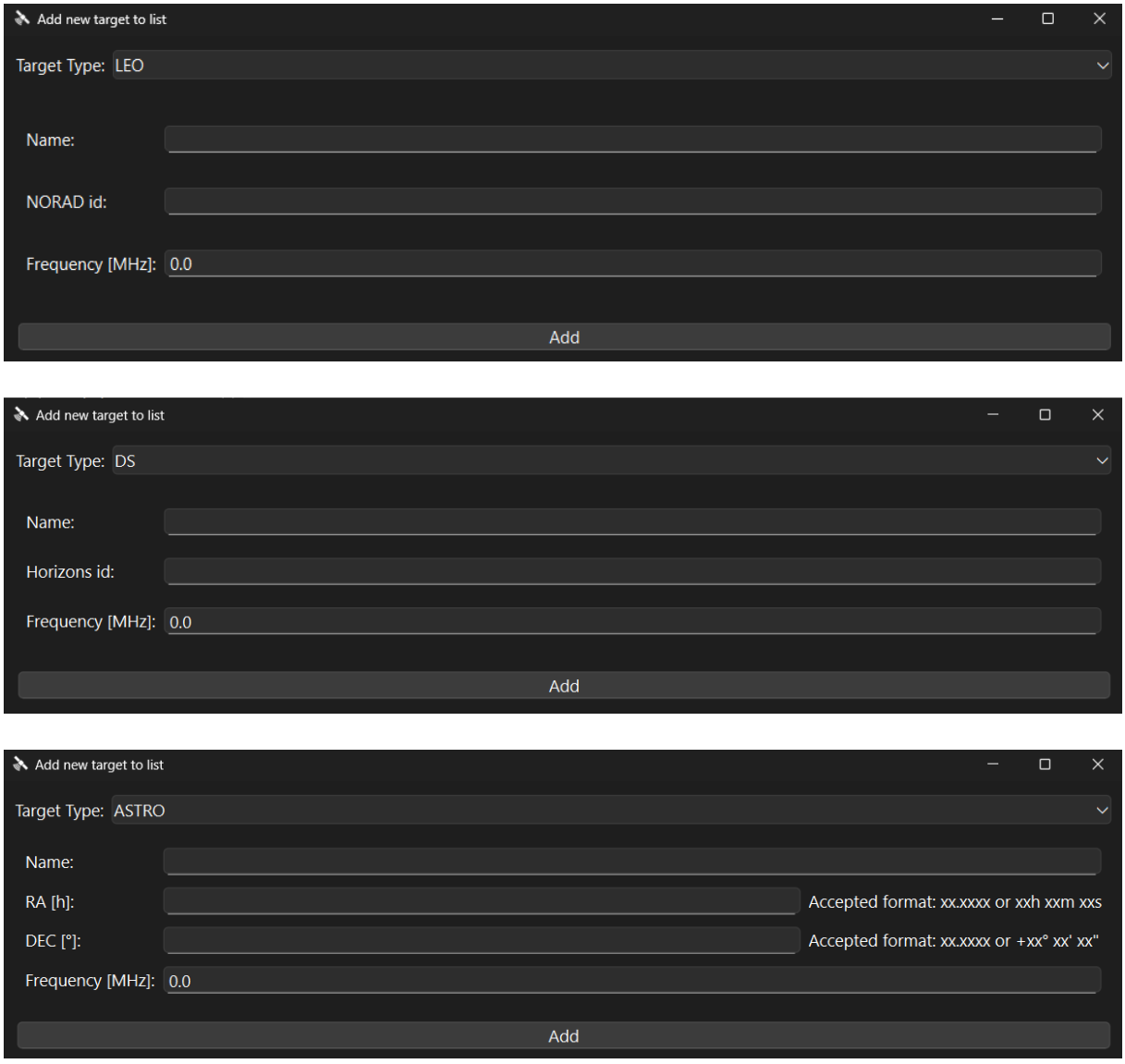



Figure 4: Interface to add new targets

### 2.3.4 Tracking Mode RA/DEC

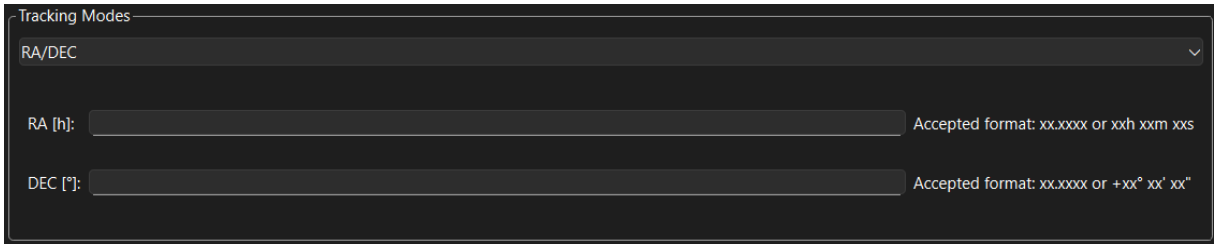



Figure 5: Tracking mode RA/DEC now accepts a more verbose format.

RA and DEC coordinates are often given like this: 02h 31m 48.7s, +89° 15’ 51”. So we support this input format now as well.

### 2.3.5 Tracking Mode OMM File

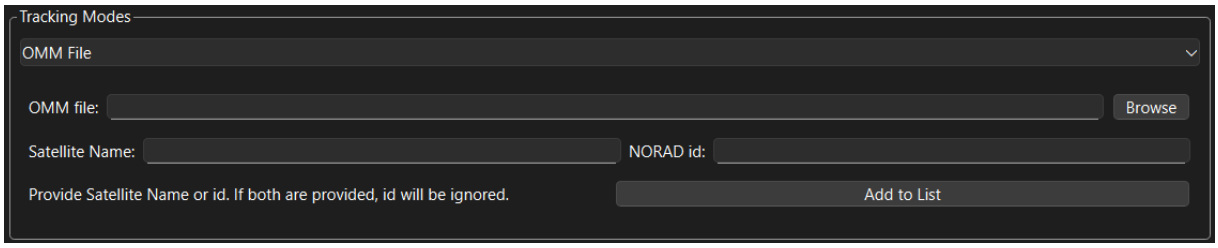



Figure 6: Caption

As discussed before, TLEs are considered deprecated, and we no longer support them. We also no longer use the international designator to find satellites in data, as the name and NORAD ID are enough.

# 3 Code Overview

The biggest change from version 1 to version 3 is that the code is now distributed over several files, instead of just one. This makes it much easier to understand and work with. However, this also generates additional overhead since now we have to deal with the fact that functions in different files need to communicate with each other. In this section, I want to give an overview of how we achieve that using the `PySide6` framework. Note that all source code is in the directory `main/src/`. All paths given in this section should be understood as being relative to `src/`, if not indicated otherwise. Also note that this section will only give you a broad overview; for more detailed documentation, look at the comments in the code itself.

## 3.1 The Core

The app is started by running `main.py`.[3] This then creates an instance of the class `AppCore`, which you can find in `core/app_core.py`. The task of the core is to start and shut down the app; to hold the three main threads (cf. Section 3.2); to make the connections between signals and slots (cf. Section 3.3); to handle the `tracking` flag (cf. Section 3.3.1); and to open temporary windows such as `visualise next pass` and `add to list`.

## 3.2 Main Threads

There are three main threads: the UI thread, the main loop thread, and the motor controller thread. Below are listed, for each thread, the instance of the class (as member variables of `AppCore`), the name of the class, and where to find the class:

- UI: `self.main_window, SatelliteTrackerApp, ui/ui_main.py`
- main loop: `self.main_loop_worker, MainLoop, core/main_loop.py`
- motor controller: `self.motor_worker, MotorWorker, utils/motor_controller.py`

Don't be confused by `self.main_loop_thread` and `self.motor_thread`, which you will also find in `AppCore`. Those only exist for `PySide6` to create the different threads. If you need to access the different threads, use the variables listed above.

[3] Alternatively to `python main/src/main.py` one can run `python satellite_tracker.py`, but this is just a wrapper.

The classes themselves are also split up and distributed over several files. Below are listed all the functions of `SatelliteTrackerApp` and where to find them:

```
src/
├── ui/
│   ├── ui_main.py
│   │   ├── __init__()
│   │   ├── log_message()
│   │   ├── SIGNALS
│   │   ├── SLOTS
│   │   ├── on_tracking_mode_changed()
│   │   ├── keyPressEvent()
│   │   ├── closeEvent()
│   │   ├── UTC_local_time_button_func()
│   │   ├── start_time_func()
│   │   └── end_time_func()
│   ├── ui_buttons.py
│   │   ├── browse_list()
│   │   ├── browse_OMM()
│   │   └── browse_spice()
│   ├── ui_setup.py
│   │   ├── set_style()
│   │   ├── setup_ui()
│   │   ├── setup_find_passes_widget()
│   │   ├── setup_tracking_modes_widget()
│   │   ├── setup_antenna_widget()
│   │   ├── setup_data_widget()
│   │   ├── setup_tracking_widget()
│   │   └── setup_status_widget()
│   └── ui_update.py
│       ├── update_ui()
│       ├── update_map()
│       └── update_ui_tracking()
└── utils/
    └── helper.py
        └── get_target_names_from_file()
```

The same for MainLoop:

- src/
  - core/
    - main_loop.py
      - __init__()
      - log_message()
      - SIGNALS
      - SLOTS
      - start_loop()
      - load_init_f0()
      - main_loop()
  - utils/
    - calculations.py
      - correction_matrix()
    - time_convertions.py
      - local_time_to_UTC()
      - utc_now()
    - tracking_modes.py
      - tracking_mode_List()
      - tracking_mode_List_core()
      - tracking_mode_RA_DEC()
      - tracking_mode_OMM()
      - tracking_mode_SPICE()
      - tracking_mode_AZ_EL()
    - helper.py
      - ra_dec_parser()
      - load_planet_ephemeris()
      - load_target_list_json()
      - load_target_list_data()
      - should_ground_track_get_calculated()
      - OMM_add_to_list()
      - find_passes()
      - visualise_next_pass()
    - get_data.py
      - save_metadata()
      - load_metadata()
      - query_celestrak_api()
      - query_horizons_api()
      - update_data_if_needed()

And for `MotorWorker`:

```
src/
└── utils/
    └── motor_controller.py
        ├── __init__()
        ├── log_message()
        ├── SIGNALS
        ├── SLOTS
        ├── establish_connection()
        ├── close_connection()
        ├── talk_to_motor_controller()
        ├── create_set_position_packet()
        └── should_update_motors()
```

For the functions in the different files to act as they would in one file, we need to import them into the main file[4] and bind them to the class. For example, like this:

```
from utils.tracking_modes import tracking_mode_List
class MainLoop(QObject):
    tracking_mode_List = tracking_mode_List

    def __init__(self, config):
        # ...
```

Each of the three classes can also access the static settings saved in `main/config/` via the `config` object passed down from `AppCore`.

## 3.3 Signals and Slots

The signal and slot system is a core feature of `PySide6`. The idea is that a function gets connected to a trigger. When the trigger is triggered, the function gets called. In this section, I will explain this concept with the help of some minimal examples.

We have a button, and we want to print `hello` when the button is pressed.

```
class myWindow(QWidget):
    def __init__(self):
        super().__init__()

        # Create button
        self.my_btn = QPushButton('say hello')
```

[4] meaning the file in which the class is defined

```
        # Connect function to trigger
        self.my_btn.clicked.connect(self.say_hello)

        # ...

    def say_hello(self):
        print('hello')
```

Some triggers also pass down an argument:

```
class myWindow(QWidget):
    def __init__(self):
        super().__init__()

        # Create text input
        self.my_text_input = QLineEdit()

        # Connect function to trigger
        self.my_text_input.textChanged.connect(self.say_msg)

        # ...

    def say_msg(self, msg):
        print(msg)
```

So far, this only works if the trigger and the function are part of the same class. But we want to connect triggers and functions in different classes.[5] For that, we need signals and slots.[6] We will make a minimal example with 3 files `ui.py`, `msg.py` and `core.py`.[7] As you can see, we create a custom signal in `ui.py` that we connect in `core.py` to the slot in `msg.py`.

```
----------------------------- ui.py -----------------------------
class myWindow(QWidget):
    msg_changed = Signal(str) # custom signal

    def __init__(self):
        super().__init__()
        self.my_text = QLineEdit()

        # connect trigger to custom signal
        self.my_text.textChanged.connect(self.msg_changed.emit)
```

[5] It also works across different threads.
[6] Technically, what we have been calling triggers and functions are already signals and slots.
[7] It is possible to do it without `core.py`, but I find it much easier having all connections in one place.

```
        # ...

---------------------------- msg.py ----------------------------
class msgHandler(QObject):
    def say_msg(self, msg): # Slot
        print(msg)

---------------------------- core.py ----------------------------
window = myWindow()
handler = msgHandler()

# connect custom signal to slot
window.msg_changed.connect(handler.say_msg)

# ...
```

Only when writing this documentation did I realize that the custom signal is actually not needed in most cases. It would be much simpler to connect the slot directly to the trigger in `core.py`[8] without the need for a custom signal.

```
---------------------------- ui.py ----------------------------
class myWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.my_text = QLineEdit()

        # ...

---------------------------- core.py ----------------------------
window = myWindow()
handler = msgHandler()

# connect trigger to slot directly
window.my_text.textChanged.connect(handler.say_msg)

# ...
```

This is something that I might change soon, so I included it in this documentation.

[8] `core/app_core.py` in the real app.

### 3.3.1 Tracking Flag

There is one signal and slot connection I want to mention specifically. All three threads need to be aware of whether we are currently tracking a target or not. They can't share a common variable, so there are three instances of a variable called `self.tracking`, one in each class. These three flags need to stay in sync at all times. The way we achieve that is that the main loop and the UI each have a function that emits a signal to the core. The core then emits three signals, updating the flag in all three threads. This way, we can guarantee that the flags stay in sync with each other.

# 4 Possible Future Features

## 4.1 Pointing Model

Currently, we are using a rotation matrix in order to correct for a tilted antenna. However, this could be replaced by a proper pointing model.

## 4.2 Schedule Mode

I was considering implementing a new tracking mode where one could schedule observations in advance. Currently, we can just wait for the next pass of the selected target, but it would be useful if we could schedule several passes for different targets. I put all the logic from `tracking_mode_List()` into `tracking_mode_List_core()`, such that a tracking mode Schedule could simply reuse this function.

## 4.3 Find Passes Data Limit

For Deep Space targets, the find passes feature can only look for passes in the time frame for which it has data. If the requested time frame exceeds the available data, it will inform the user. However, it would be better if the find passes feature were able to download the missing data.

## 4.4 Atmospheric Conditions

Currently, we are ignoring the effect of atmospheric conditions on the apparent azimuth and elevation. The functions that we are using for the calculations of azimuth and elevation are all capable of including this. We would need to build an interface, or maybe even an API, with which we can feed the needed data into the relevant functions.

## 4.5 Edit Targets

Currently, we have a button to add new targets to the list, but no button to remove or edit them. This might be useful.

## 4.6 Difference Between Observer and Vector Data

As discussed above, we observe a periodic difference between the values that we get from Horizons directly (observer data) and the values that we calculate from state vectors. Figure 7 shows the absolute difference for the whole duration of the Artemis II mission. While we work around that problem by using the observer data, it would be good to figure out what exactly is going on. I have two guesses. And I need to stress that they are only guesses. I could be completely wrong.

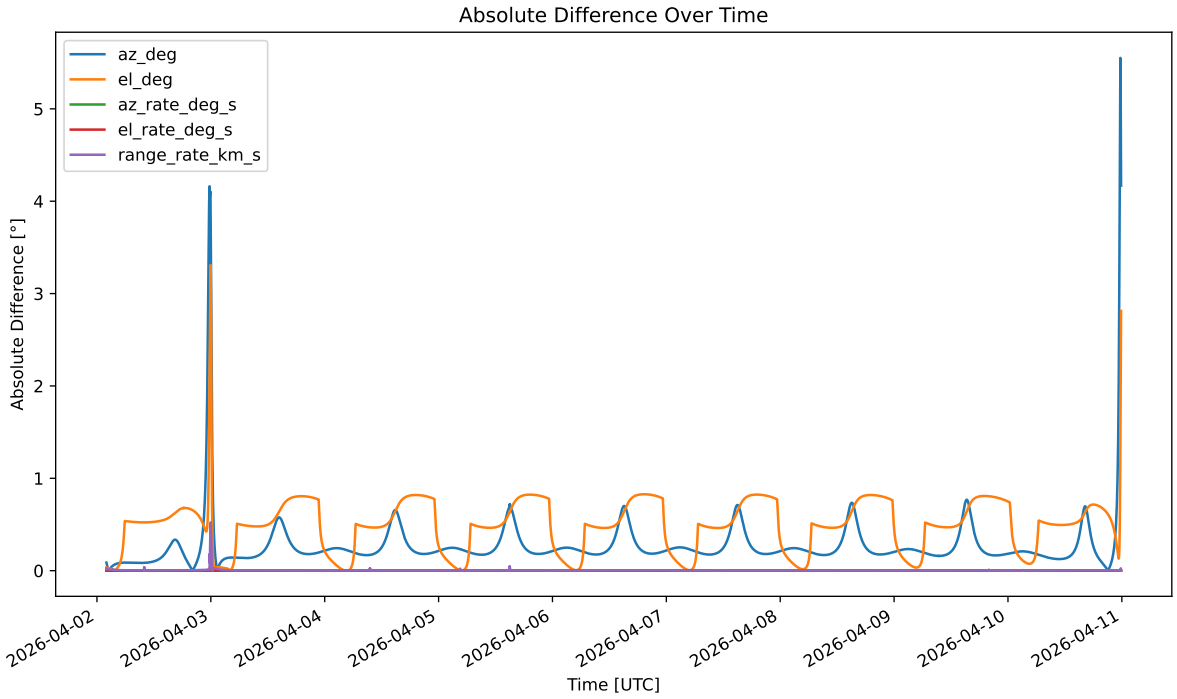



Figure 7: The difference between observer (direct) and vector data is periodic with a period of one day, indicating some systematic error.

The best hint we have is that the pattern is very regular with a period of one day. So my first guess is that it has something to do with the geometry of the situation. Something about the fact that the antenna rotates around the Earth's axis. My other guess is that something went wrong with the conversion of time.

One problem I had to deal with again and again was that the time was given in a different format. Not only do we have to deal with the fact that all the different packages have their own time object, and that there are way too many ways to represent the same time as different strings, but also that there are way too many ways of measuring time.

For example, the data we get from the Horizons API is given in the JD format with TDB scale. If you use the Horizons website, it will give you the time in UT. If you ask the API for vector data at a point in time where they don't have any data, it replies with an error message containing a time in TD. However, if you ask the same but for observer data, it will reply with time in U. By the way, this website[9] explains some (but not all) of these acronyms.

As you can see, it is a mess. So, there is a decent chance that somewhere, some conversion from one into the other was not done properly, resulting in the data getting out of sync.

However, none of this could explain the two massive spikes. The spike at the end could

[9]https://www.cnmoc.usff.navy.mil/Our-Commands/United-States-Naval-Observatory/Precise-Time-Department/The-USNO-Master-Clock/Definitions-of-Systems-of-Time/

maybe be associated with the reentry of the spacecraft. Small differences in how Horizons processes data and how we do it could maybe have a big impact on the resulting azimuth and elevation values. However, this is just a gut feeling. I don't even know if the data covers the reentry itself.

It would also be a good idea to do this kind of comparison for multiple missions and spacecraft in very different orbits, to check if they share the same or any periodic pattern.